


```
input_file = "Conformer3D_COMPOUND_CID_2519.sdf"
mols = Chem.SDMolSupplier(input_file, removeHs=False)
mols = [m for m in mols if m is not None]
```

```
print("Molecules loaded:", len(mols))
```

```
Molecules loaded: 1
```

2 Generate 3D Conformers and Energy Minimization

```
from rdkit.Chem import AllChem
```

```
mol = mols[0]
mol3d = Chem.AddHs(mol)
```

```
print("Embedding conformers...")
conf_ids = AllChem.EmbedMultipleConfs(
    mol3d,
    numConfs=20,
    randomSeed=42
)
```

```
print("Total conformers generated:", len(conf_ids))
```

```
Embedding conformers...
Total conformers generated: 20
```

```
energies = []
```

```
print("Minimizing conformers...")
```

```
for cid in conf_ids:
    try:
        props = AllChem.MMFFGetMoleculeProperties(mol3d)
        ff = AllChem.MMFFGetMoleculeForceField(mol3d, props, confId=cid)
        ff.Minimize()
        energy = ff.CalcEnergy()
    except:
        ff = AllChem.UFFGetMoleculeForceField(mol3d, confId=cid)
        ff.Minimize()
        energy = ff.CalcEnergy()
```

```
    energies.append((cid, energy))
```

```
print("")
```

```
print("Minimization complete.")
```

```
for cid, e in energies:
    print(f"Conformer {cid}: Energy = {e:.4f}")
```

```
Minimizing conformers...
```

```
Minimization complete.
Conformer 0: Energy = -122.5284
Conformer 1: Energy = -122.5284
Conformer 2: Energy = -122.5284
Conformer 3: Energy = -122.5284
Conformer 4: Energy = -122.5284
Conformer 5: Energy = -122.5284
Conformer 6: Energy = -122.5284
Conformer 7: Energy = -122.5284
Conformer 8: Energy = -122.5284
Conformer 9: Energy = -122.5284
Conformer 10: Energy = -122.5284
Conformer 11: Energy = -122.5284
Conformer 12: Energy = -122.5284
Conformer 13: Energy = -122.5284
Conformer 14: Energy = -122.5284
Conformer 15: Energy = -122.5284
Conformer 16: Energy = -122.5284
Conformer 17: Energy = -122.5284
Conformer 18: Energy = -122.5284
Conformer 19: Energy = -122.5284
```

Explanation:

Multiple 3D conformers are generated to represent different possible geometries.

Energy minimization ensures stable conformations using MMFF first, then UFF if MMFF is unavailable.

3 Score Conformers and Filter by Energy

```
import numpy as np

energies_array = np.array([e for _, e in energies])
min_energy = energies_array.min()
energy_window = 5.0

good_conformers = []

for cid, energy in energies:
    if energy <= min_energy + energy_window:
        good_conformers.append((cid, energy))

good_conformers
```

```
[(0, -122.52841057522076),
 (1, -122.52841057495584),
 (2, -122.52841057516962),
 (3, -122.52841057522609),
 (4, -122.52841057528585),
 (5, -122.52841057464775),
 (6, -122.52841057573947),
 (7, -122.52841057564356),
 (8, -122.52841057517732),
 (9, -122.52841057456651),
 (10, -122.52841057432848),
 (11, -122.52841057375932),
 (12, -122.52841057470653),
 (13, -122.52841057564689),
 (14, -122.52841057542395),
 (15, -122.52841057530439),
 (16, -122.52841057492422),
 (17, -122.52841057552074),
 (18, -122.52841057547326),
 (19, -122.52841057473037)]
```

Explanation:

Calculates energy for each conformer.

Filters low-energy conformers for further pharmacophore analysis. low-energy conformers (Stable)

4 Extract Pharmacophore Features

```
from rdkit.Chem import ChemicalFeatures
from rdkit import RDConfig
import os

fdef_path = os.path.join(RDConfig.RDDataDir, 'BaseFeatures.fdef')
factory = ChemicalFeatures.BuildFeatureFactory(fdef_path)

all_features = {}

for cid, energy in good_conformers:
    feats = factory.GetFeaturesForMol(mol3d, confId=cid)
    all_features[cid] = feats

all_features
```

```
{0: (<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d070>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d2a0>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d310>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d380>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d3f0>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d460>),
 1: (<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d4d0>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d540>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d5b0>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d620>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d690>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d700>),
 2: (<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d770>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d7e0>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d850>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d8c0>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d930>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0d9a0>),
 3: (<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0da10>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0da80>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0daf0>,
 <rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0db60>),
```

```

<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0dbd0>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0dc40>),
4: (<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0dcb0>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0dd20>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0dd90>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0de00>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0de70>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0dee0>),
5: (<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0df50>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0dfc0>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e030>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e0a0>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e110>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e180>),
6: (<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e1f0>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e260>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e2d0>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e340>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e3b0>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e420>),
7: (<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e490>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e500>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e570>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e5e0>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e650>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e6c0>),
8: (<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e730>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e7a0>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e810>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e880>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e8f0>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e960>),
9: (<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0e9d0>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0ea40>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0eab0>,
<rdkit.Chem.rdMolChemicalFeatures.MolChemicalFeature at 0x7887b0b0eb20>,

```

Explanation:

Extracts pharmacophore features like hydrogen bond donors, acceptors, and aromatic rings using RDKit's predefined features.

5 Compute Feature Coordinates

```

from rdkit.Geometry import Point3D

def get_feature_point(feature, mol, cid):
    atom_ids = feature.GetAtomIds()
    conf = mol.GetConformer(cid)

    if len(atom_ids) == 1:
        idx = atom_ids[0]
        pos = conf.GetAtomPosition(idx)
        return (pos.x, pos.y, pos.z)

    xs = []
    ys = []
    zs = []
    for idx in atom_ids:
        pos = conf.GetAtomPosition(idx)
        xs.append(pos.x)
        ys.append(pos.y)
        zs.append(pos.z)

    return (sum(xs)/len(xs), sum(ys)/len(ys), sum(zs)/len(zs))

feature_coords = {}

for cid, feats in all_features.items():
    coords = []
    for f in feats:
        coords.append({
            "type": f.GetType(),
            "point": get_feature_point(f, mol3d, cid)
        })
    feature_coords[cid] = coords

feature_coords

```

```

    {'type': 'SingleAtomAcceptor',
     'point': (0.7134711430712157, -2.2390982152800376, -0.38918032253074325)},
    {'type': 'Imidazole',
     'point': (1.1206659478306207, -1.1352283414000388, -0.2717323926058792)},
    {'type': 'Arom5',
     'point': (1.1206659478306207, -1.1352283414000388, -0.2717323926058792)}],
16: [{'type': 'SingleAtomDonor',
     'point': (1.5445659820165993, -0.7591436237055151, 0.2742120955832425)},
    {'type': 'SingleAtomAcceptor',
     'point': (-0.00144425970168017, -2.317671995620241, 1.039616224092973)},
    {'type': 'SingleAtomAcceptor',
     'point': (3.041290898659546, 0.8131448539907006, -0.49524603821424834)},
    {'type': 'SingleAtomAcceptor',
     'point': (-1.6189227002227715, 1.6191942244865434, -0.6567810468962378)},
    {'type': 'Imidazole',
     'point': (-1.5138834915821258, 0.5429120940612104, -0.1785408166683937)},
    {'type': 'Arom5',
     'point': (-1.5138834915821258, 0.5429120940612104, -0.1785408166683937)}],
17: [{'type': 'SingleAtomDonor',
     'point': (-1.3057305675380095, 1.0858915812088836, -0.3912567631914766)},
    {'type': 'SingleAtomAcceptor',
     'point': (0.596540719069314, 2.4180416631715236, -0.49962041497446474)},
    {'type': 'SingleAtomAcceptor',
     'point': (-3.1636498254989425, -0.26998957093583126, -0.27266905818327336)},
    {'type': 'SingleAtomAcceptor',
     'point': (1.1571280275518765, -1.9974317823198133, 0.5876944433488094)},
    {'type': 'Imidazole',
     'point': (1.3314445433934279, -0.8544337622947331, 0.340106740216341)},
    {'type': 'Arom5',
     'point': (1.3314445433934279, -0.8544337622947331, 0.340106740216341)}],
18: [{'type': 'SingleAtomDonor',
     'point': (-1.1429150033557918, -1.312536426522781, -0.15753941674896)},
    {'type': 'SingleAtomAcceptor',
     'point': (0.9243015697800254, -2.3342129837848797, -0.4538419184532435)},
    {'type': 'SingleAtomAcceptor',
     'point': (-3.1694943465484235, -0.25952126307569995, 0.13991062311810967)},
    {'type': 'SingleAtomAcceptor',
     'point': (0.8699973588254619, 2.2052874975320127, 0.16494984529156378)},
    {'type': 'Imidazole',
     'point': (1.2003141237902621, 1.0838322544031072, -0.01213603413925259)},
    {'type': 'Arom5',
     'point': (1.2003141237902621, 1.0838322544031072, -0.01213603413925259)}],
19: [{'type': 'SingleAtomDonor',
     'point': (-1.1095684154925973, 1.3417123729204659, 0.07642088762731511)},
    {'type': 'SingleAtomAcceptor',
     'point': (0.9902829032582979, 2.328206882940803, 0.2262692971233619)},
    {'type': 'SingleAtomAcceptor',
     'point': (-3.169549624635098, 0.32291987382354026, -0.07484486919081602)},
    {'type': 'SingleAtomAcceptor',
     'point': (0.8115980462207792, -2.233744184207615, -0.16049760266498972)},
    {'type': 'Imidazole',
     'point': (1.1733579939716319, -1.112947224913651, -0.05526401346294203)},
    {'type': 'Arom5',
     'point': (1.1733579939716319, -1.112947224913651, -0.05526401346294203)}],

```

Explanation:

Coordinates of pharmacophore features are calculated for visualization and clustering.

6 Compute Pairwise Feature Distances

```

import math

def distance(p1, p2):
    return math.sqrt(
        (p1[0] - p2[0])**2 +
        (p1[1] - p2[1])**2 +
        (p1[2] - p2[2])**2
    )

max_cutoff = 7.0

pairwise_distances = {}

for cid, feats in feature_coords.items():
    dist_list = []
    for i in range(len(feats)):
        for j in range(i+1, len(feats)):
            p1 = feats[i]["point"]
            p2 = feats[j]["point"]
            d = distance(p1, p2)
            if d <= max_cutoff:
                dist_list.append({
                    "feat1": feats[i]["type"],
                    "feat2": feats[j]["type"],
                    "distance": d
                })

```

```

        distance = d
    })
    pairwise_distances[cid] = dist_list

pairwise_distances

{'feat2': 'SingleAtomAcceptor',
 'distance': 4.732166779267189},
{'feat1': 'SingleAtomAcceptor',
 'feat2': 'Imidazole',
 'distance': 4.574160352059113},
{'feat1': 'SingleAtomAcceptor',
 'feat2': 'Arom5',
 'distance': 4.574160352059113},
{'feat1': 'SingleAtomAcceptor',
 'feat2': 'Imidazole',
 'distance': 1.1824256578119021},
{'feat1': 'SingleAtomAcceptor',
 'feat2': 'Arom5',
 'distance': 1.1824256578119021},
{'feat1': 'Imidazole', 'feat2': 'Arom5', 'distance': 0.0}],
19: [{'feat1': 'SingleAtomDonor',
      'feat2': 'SingleAtomAcceptor',
      'distance': 2.3248659154860163},
     {'feat1': 'SingleAtomDonor',
      'feat2': 'SingleAtomAcceptor',
      'distance': 2.3031157303579253},
     {'feat1': 'SingleAtomDonor',
      'feat2': 'SingleAtomAcceptor',
      'distance': 4.065821016297308},
     {'feat1': 'SingleAtomDonor',
      'feat2': 'Imidazole',
      'distance': 3.3547649165691977},
     {'feat1': 'SingleAtomDonor',
      'feat2': 'Arom5',
      'distance': 3.3547649165691977},
     {'feat1': 'SingleAtomAcceptor',
      'feat2': 'SingleAtomAcceptor',
      'distance': 4.627748090617022},
     {'feat1': 'SingleAtomAcceptor',
      'feat2': 'SingleAtomAcceptor',
      'distance': 4.58180253306087},
     {'feat1': 'SingleAtomAcceptor',
      'feat2': 'Imidazole',
      'distance': 3.4575018565161986},
     {'feat1': 'SingleAtomAcceptor',
      'feat2': 'Arom5',
      'distance': 3.4575018565161986},
     {'feat1': 'SingleAtomAcceptor',
      'feat2': 'SingleAtomAcceptor',
      'distance': 4.732166974393561},
     {'feat1': 'SingleAtomAcceptor',
      'feat2': 'Imidazole',
      'distance': 4.574160504278285},
     {'feat1': 'SingleAtomAcceptor',
      'feat2': 'Arom5',
      'distance': 4.574160504278285},
     {'feat1': 'SingleAtomAcceptor',
      'feat2': 'Imidazole',
      'distance': 1.1824255545511695},
     {'feat1': 'SingleAtomAcceptor',
      'feat2': 'Arom5',
      'distance': 1.1824255545511695},
     {'feat1': 'Imidazole', 'feat2': 'Arom5', 'distance': 0.0}]]

```

Explanation:

Calculates spatial relationships between features using Euclidean distance.

7 Build Consensus Pharmacophore

```

import math

def dist(p1, p2):
    return math.sqrt(
        (p1[0]-p2[0])**2 +
        (p1[1]-p2[1])**2 +
        (p1[2]-p2[2])**2
    )

same_type_clusters = {}
threshold = 1.5

for cid, feats in feature_coords.items():
    for feat in feats:

```

```

ftype = feat["type"]
point = feat["point"]

if ftype not in same_type_clusters:
    same_type_clusters[ftype] = []

placed = False
for cluster in same_type_clusters[ftype]:
    if dist(point, cluster["center"]) < threshold:
        cluster["points"].append(point)
        xs = [p[0] for p in cluster["points"]]
        ys = [p[1] for p in cluster["points"]]
        zs = [p[2] for p in cluster["points"]]
        cluster["center"] = (
            sum(xs)/len(xs),
            sum(ys)/len(ys),
            sum(zs)/len(zs)
        )
        placed = True
        break

if not placed:
    same_type_clusters[ftype].append({
        "points": [point],
        "center": point
    })

consensus_pharmacophore = {}

for ftype, clusters in same_type_clusters.items():
    consensus_pharmacophore[ftype] = [
        cluster["center"] for cluster in clusters
    ]

consensus_pharmacophore

{'SingleAtomDonor': [(-1.062771384571464,
1.3376327326726305,
-0.0751342777082694),
(-1.3282120419123302, -1.100483323328049, -0.020122534738302455),
(1.5445659820165993, -0.7591436237055151, 0.2742120955832425)],
'SingleAtomAcceptor': [(1.1037467614950374,
2.165679948302828,
-0.07830255927427399),
(-3.1300525874269054, 0.33370310972008843, -0.025680284665006665),
(0.6248845022444525, -2.311746152376298, 0.07007050586318472),
(3.041290898659546, 0.8131448539907006, -0.49524603821424834),
(-1.6189227002227715, 1.6191942244865434, -0.6567810468962378)],
'Imidazole': [(1.13108375691517, -1.114197921607843, 0.017645315380625356),
(1.3480616419998197, 0.8656423394550851, -0.038392105062091626),
(-1.5138834915821258, 0.5429120940612104, -0.1785408166683937)],
'Arom5': [(1.13108375691517, -1.114197921607843, 0.017645315380625356),
(1.3480616419998197, 0.8656423394550851, -0.038392105062091626),
(-1.5138834915821258, 0.5429120940612104, -0.1785408166683937)]]

```

Explanation:

Consolidates features by type across all conformers to form a consensus pharmacophore.

8 Export to JSON

```

mol_name = "Conformer3D_COMPOUND_CID_2519"
import json

output = {
    "molecule": mol_name,
    "consensus_pharmacophore": consensus_pharmacophore
}

with open("pharmacophore_output.json", "w") as f:
    json.dump(output, f, indent=4)

"JSON file saved as pharmacophore_output.json"

'JSON file saved as pharmacophore_output.json'

```

Explanation:

Saves all pharmacophore data in JSON format for downstream analysis.

